

课程笔记

vLLM 推理优化技术全景与 AI-Infra 学习路径

五道口纳什 & Codex

2026-07-18



视频作者/频道: Bilibili: 五道口纳什

发布日期: 2026-07

视频时长: 约 2 小时 46 分钟

视频链接: <https://www.bilibili.com/video/BV1FCLV6LEsu/>

目录

1 vLLM 资源与生态	2
1.1 本章小结	2
2 vLLM 推理优化技术全景	2
2.1 计算优化	2
2.2 内存优化	2
2.3 分布式并行	3
2.4 调度优化	3
2.5 架构层面	3
2.6 本章小结	4
3 vLLM 架构总览	4
3.1 自上而下的分层架构	4
3.2 数据并行与 Engine Core 的关系	5
3.3 注意力后端	6
3.4 本章小结	7
4 内存管理优化: KV Cache 与 PagedAttention	7
4.1 KV Cache 的必要性	7
4.2 PagedAttention 原理	8
4.3 前缀缓存	9
4.4 本章小结	10
5 分布式并行	10
5.1 集合通信基础	10
5.2 数据并行 (Data Parallelism, DP)	11
5.3 专家并行 (Expert Parallelism, EP)	12
5.4 张量并行 (Tensor Parallelism, TP)	13
5.5 上下文并行 (Context Parallelism, CP)	14
5.6 本章小结	15
6 量化压缩	15
6.1 量化方法概览	15
6.2 vLLM 量化设计原理	16
6.3 量化粒度	17
6.4 本章小结	18
7 投机解码	18
7.1 投机解码的动机	18
7.2 Medusa 方案	18
7.3 Eagle 方案	19
7.4 MTP 方案	20
7.5 本章小结	20

8 AI-Infra 学习路径与职业发展（答疑精选）	21
8.1 从零开始学 AI-Infra	21
8.2 转行与面试准备	21
8.3 行业现状与趋势	21
8.4 本章小结	22
9 总结与延伸	22
9.1 讲者结尾讨论	22
9.2 核心知识综合	22
9.3 实践要点	23
9.4 拓展阅读	23

1 vLLM 资源与生态

vLLM 作为一个快速迭代的开源推理框架，学习它的第一步不是读源码，而是搞清楚它的信息入口在哪里。讲者将 vLLM 的生态资源分为几类：

- **官方主站** (docs.vllm.ai)：项目入口，包含核心特性介绍和 Quick Start。
- **技术博客** (Blog)：当 vLLM 发布重大新特性时，会以博客形式做深度技术解读。例如 PagedAttention、Prefix Caching、V1 引擎等重大更新均有对应的博客文章。
- **Release Notes**：vLLM 更新极快，每个版本都有详细的 Release Notes 列出新特性、Bug 修复和 Breaking Changes。做模型支持的同学关注模型适配部分，做 Engine 开发的同学关注核心改动。
- **代码仓库示例** ([examples/](#) 目录)：提供离线推理和在线服务的实例代码，是新手入门的最佳起点。
- **GitHub Issues & PR**：跟踪社区动态，了解最新讨论和未解决的问题。

学习建议

如果你是做模型支持的，主要关注 Release Notes 中的模型适配部分；如果做 Engine 开发，核心关注架构层面的更新和调度优化。每隔一段时间要刷一遍 Release Notes 了解最新进展。

1.1 本章小结

vLLM 生态围绕官方文档、博客、Release Notes 和 GitHub 展开。新人应先从主站和 [examples/](#) 入手建立直觉，再根据自身方向选择性地深入对应模块的代码和文档。

2 vLLM 推理优化技术全景

讲者将 vLLM 所运用的推理优化技术归纳为一张全景图，从计算、内存、分布式、调度、架构五个维度进行分类。

2.1 计算优化

- **算子融合** (Kernel Fusion)：将多个小算子合并为一个 Kernel，减少显存读写和 Kernel Launch 开销。典型如 FlashAttention 将 attention 中的多个操作融合。
- **定制化 Kernel**：针对特定硬件（如 CUDA Core、Tensor Core）和特定算子（如激活函数、LayerNorm）手写高性能 Kernel。

2.2 内存优化

- **PagedAttention**：vLLM 的核心创新，将操作系统中的分页内存管理思想引入 KV Cache 管理，实现近乎零浪费的显存利用率。
- **前缀缓存** (Prefix Caching)：基于 Hash 的 Block 级别 KV Cache 复用，跨请求共享前缀计算结果，减少 Prefill 阶段重复计算。

- **量化压缩**：通过降低权重精度减少显存占用。包括 AWQ、GPTQ（仅权重量化）和 FP8（权重 + 激活 + KV Cache 全链路量化）。

2.3 分布式并行

- **数据并行 (DP)**：将模型完整复制到多个 GPU，每个 GPU 处理独立请求批次，提升吞吐量。
- **张量并行 (TP)**：将单个权重张量按 hidden_size 维度拆分到多张 GPU，适合单层计算量大的场景。
- **专家并行 (EP)**：针对 MoE 模型，每个 GPU 只负责部分专家计算，通过 All-to-All 通信完成 token 路由。
- **上下文并行 (CP)**：将长序列按 token 维度拆分，解决超长上下文推理的内存瓶颈。

2.4 调度优化

- **Continuous Batching**：动态地将新请求加入正在执行的批次，相比 Static Batching 显著提升 GPU 利用率。
- **PD 分离 (Prefill-Decode Disaggregation)**：将 Prefill 和 Decode 阶段部署在不同 GPU 上，各自采用最优并行策略。
- **负载均衡**：DP 模式下提供内部负载均衡、混合负载均衡和外部负载均衡三种模式。

2.5 架构层面

- **V1 引擎**：vLLM 的新一代推理引擎，引入更高效的调度和 KV Cache 管理机制。
- **Multiprocessing 架构**：通过多进程管理多 GPU，支持 DP/TP/EP 等并行策略的组合使用。

核心脉络

vLLM 的优化技术并非孤立存在，而是围绕“显存效率”和“计算效率”两条主线展开。PagedAttention 解决 KV Cache 显存碎片问题，Continuous Batching 提升 GPU 计算利用率，分布式并行解决单卡算力不足的问题，量化压缩在全链路降低显存占用。理解这些技术之间的协同关系比记住每一个技术细节更重要。

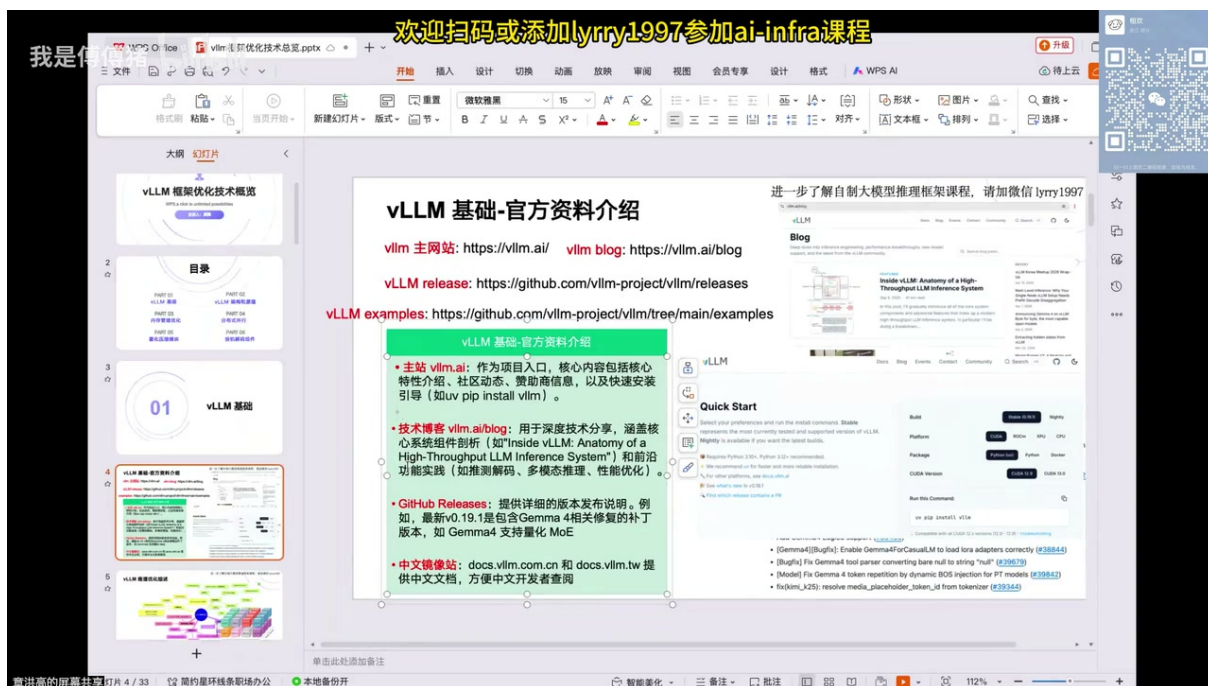


图 1: vLLM 推理优化技术全景图¹

2.6 本章小结

vLLM 的推理优化技术可以从计算、内存、分布式、调度和架构五个维度理解。PagedAttention 和 Continuous Batching 是两个最具代表性的优化技术，前者解决显存效率问题，后者解决计算效率问题。本章建立的整体框架为后续各章节的深入讨论提供了脉络。

3 vLLM 架构总览

3.1 自上而下的分层架构

从用户接口到 GPU 模型层，vLLM 的系统架构可以划分为六个层次：

1. **LLM / AsyncLLM 类**（用户入口层）：LLM 类用于离线批量推理，AsyncLLM 类用于在线服务（支持异步和流式）。两者都通过 Engine 做请求处理。
2. **Engine Core**（推理引擎进程）：以后台进程形式运行，内部包含 Scheduler（调度器），负责从请求队列中选择哪些请求进入本轮推理计算。
3. **Scheduler**（调度器）：嵌在 Engine Core 中，负责请求的批处理调度和资源分配（包括 KV Cache 内存的分配）。同时持有 KV Cache Manager。
4. **Executor**（执行层）：管理分布式并行的抽象层。将 Scheduler 的输出广播给所有 Worker，并聚合返回的模型输出。在开启 TP 后，一个 Executor 管理多个 Worker 进程。
5. **Worker**（工作进程）：每个 GPU 对应一个 Worker 进程，负责该 GPU 上的模型推理计算。

¹ 视频画面时间区间：00:03:10-00:03:20。

6. **Model** (模型对象)：放在 Worker 内部，包含具体的模型结构和注意力模块。



为什么需要这么多层抽象？

每一层抽象都是为了支持推理加速的某个功能：数据并行、张量并行、调度策略、请求路由等。这些功能不能简单摊在一个大类里完成——分层让每个模块职责清晰，也方便独立演进和替换。但讲者也指出，未来这些抽象层是否会精简或重组，目前还没有定论。

3.2 数据并行与 Engine Core 的关系

当开启数据并行 (DP) 后，vLLM 会启动多个 Engine Core 进程 (每个对应一个 DP Rank)。每个 Engine Core 内部都包含独立的 Scheduler 和 Executor (MultiProcExecutor)，形成多副本结构：

- 每个 DP Rank 拥有完整的模型权重副本
- 不同 DP Rank 通过 ZMQ 与前端通信，获取不同的请求任务
- 每个 DP Rank 内部的 MultiProcExecutor 管理本 Rank 内的 TP Worker

以 DP=2、TP=4 为例，总共 8 个 GPU：系统启动 2 个 Engine Core 进程 (2 个 DP Rank)，每个 Engine Core 管理 4 个 GPU Worker (TP=4)。这 4 个 GPU Worker 共同完成一个请求的推理，而两个 DP Rank 独立处理不同的请求组。

² 视频画面时间区间：00:07:50-00:08:10。

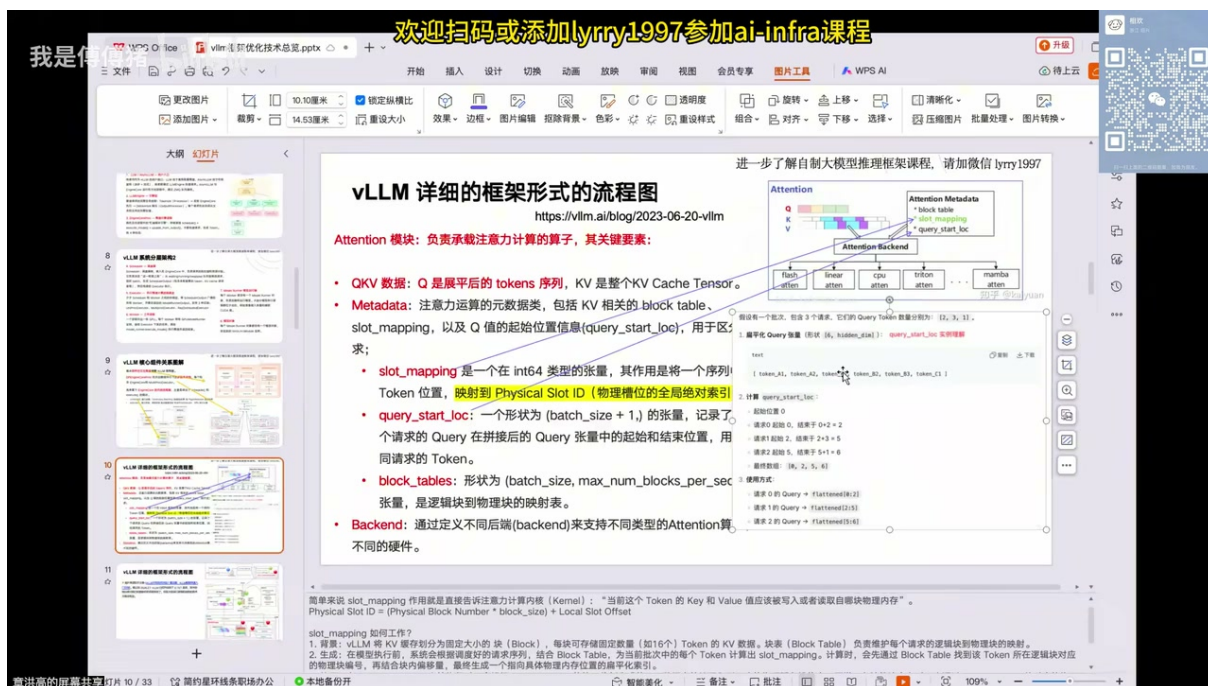


图 3: DP=2, TP=4 时的进程结构示意图³

3.3 注意力后端

注意力模块是 Transformer 模型中最重要的计算模块之一。vLLM 支持多种注意力后端：

- **FlashAttention**：目前使用最广泛的高效注意力实现，通过分块计算和 IO 优化减少 HBM 访问。
- **线性注意力**：将注意力复杂度从 $O(n^2)$ 降到 $O(n)$ ，适合超长序列。
- **CPU Attention**：用于 CPU 推理场景。
- **Mamba Attention**：状态空间模型（SSM）的注意力变体。

不同的注意力后端通过统一的接口接入 vLLM 框架，用户可以根据硬件和模型特性选择合适的实现。

³视频画面时间区间：00:17:50-00:18:10。

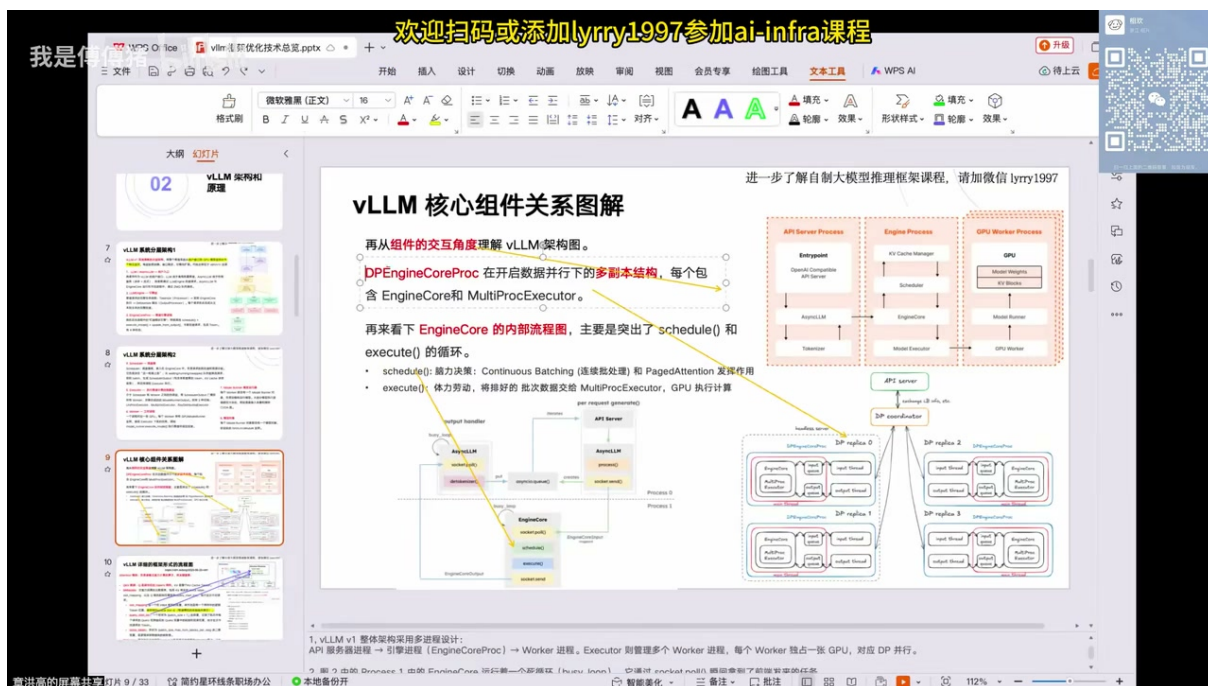


图 4: vLLM 架构数据流转示意 (引自知乎)⁴

3.4 本章小结

vLLM 的六层架构 (LLM → Engine Core → Scheduler → Executor → Worker → Model) 形成了一个清晰的请求处理流水线。数据并行的引入使得系统在多 GPU 场景下呈现多副本结构，每个 Engine Core 独立处理一组请求。分层抽象的核心价值在于让不同的优化技术能在各自的层面独立演进，同时也增加了系统的复杂度。

4 内存管理优化：KV Cache 与 PagedAttention

4.1 KV Cache 的必要性

在自回归生成过程中，每个新 token 的注意力计算需要用到之前所有 token 的 Key 和 Value。如果没有 KV Cache，每生成一个新 token 都要重新计算所有历史 token 的 K 和 V，带来极大的计算冗余。

⁴ 视频画面时间区间：00:13:50–00:14:10。

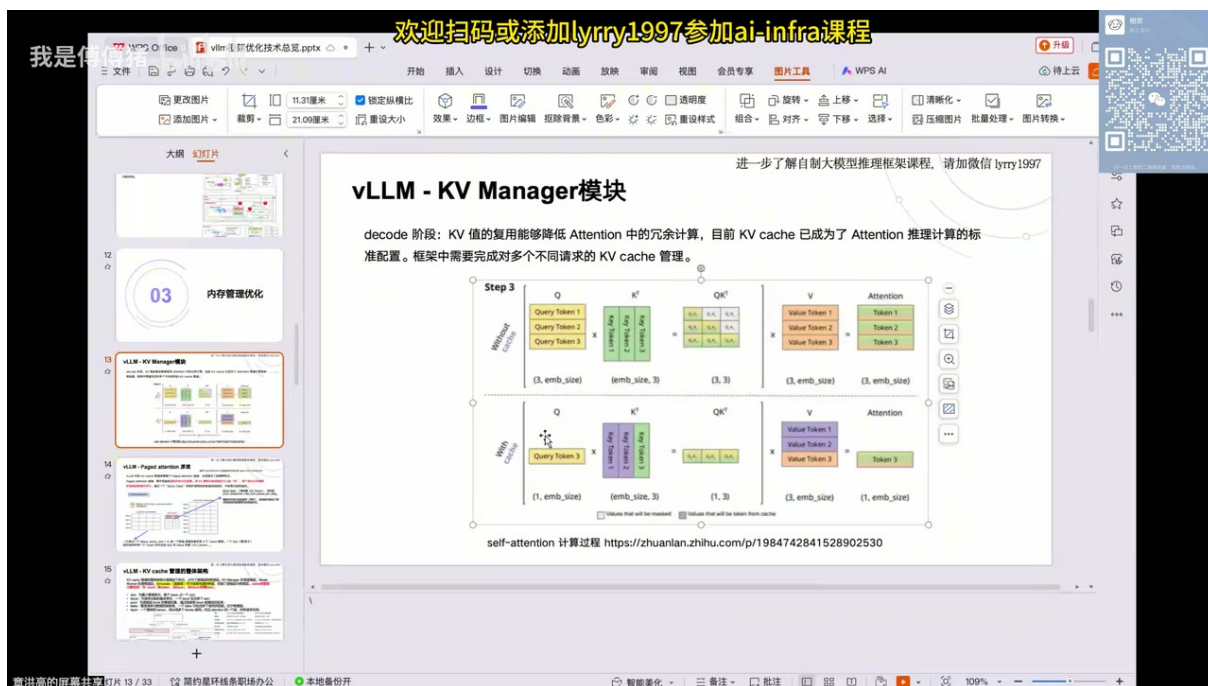


图 5: 有无 KV Cache 的计算量对比⁵

有了 KV Cache 之后：每个 token 的 K 和 V 只计算一次并缓存起来，后续 token 只需计算当前 token 的 Q 并对缓存的 K、V 做注意力运算。这大幅减少了计算量，但也引入了显存管理的挑战——不同请求的序列长度不同，预分配固定大小的 KV Cache 空间会造成大量浪费。

4.2 PagedAttention 原理

PagedAttention 是 vLLM 最核心的技术创新，其灵感来源于操作系统的虚拟内存分页机制。核心思想是将 KV Cache 划分为固定大小的 Block，通过 Block Table 实现逻辑块到物理块的映射。

关键数据结构：

- **Block**: KV Cache 的基本管理单元。一个 Block 内包含固定数量 token (block_size, 例如 16) 的 K 和 V 张量。
- **Slot**: Block 内的最小单位，对应一个 token 的 KV Cache。
- **Block Table**: 二维张量，横坐标是请求 ID，纵坐标是该请求对应的物理块号序列。实现逻辑块到物理块的映射。
- **Slot Mapping**: 一维张量，将序列中的逻辑 token 位置映射到物理 Slot 的绝对索引 (token 级别的映射)。
- **Query Start Loc**: 记录每个请求在拼接后的 Query 张量中的起始和结束位置。

⁵ 视频画面时间区间：00:20:50–00:21:10。

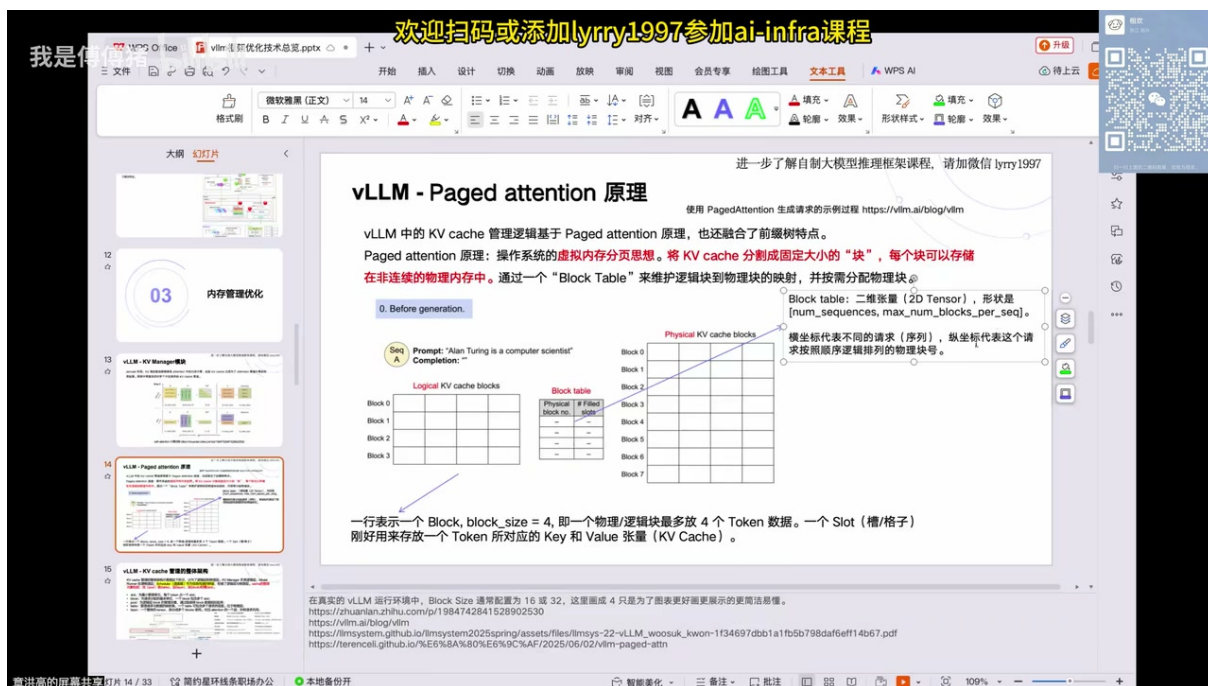


图 6: PagedAttention 逻辑块与物理块映射⁶

PagedAttention 核心机制

PagedAttention 将 KV Cache 物理存储划分为固定大小的 Block，按需分配物理块，通过 Block Table 维持逻辑块到物理块的映射。这使得不同请求的 KV Cache 可以非连续存储，避免了预分配固定空间造成的内存浪费，显存利用率接近最优。从论文角度看，PagedAttention 是基于操作系统分页思想在 LLM 推理场景中的创造性应用。

KV Cache 管理架构：在 vLLM 代码中，Scheduler 初始化时会创建并持有 KVCacheManager。KVCacheManager 负责逻辑层和物理层的 Block 管理，包括 Block 的分配、回收以及 Block Table 的维护。

Slot Mapping vs Block Table

Slot Mapping 和 Block Table 容易混淆，但它们是不同粒度的映射：

- Slot Mapping 是 **token 级别**的映射，将每个逻辑 token 位置映射到物理 Slot 索引。
- Block Table 是 **Block 级别**的映射，将逻辑块映射到物理块。

两者的 Shape 也不同，在实际代码中需要区分清楚。

4.3 前缀缓存

除了 PagedAttention 的物理内存管理优化，vLLM 还通过前缀缓存 (Automatic Prefix Caching) 实现 KV Cache 的逻辑复用。当多个请求共享相同的前缀（如 System Prompt）时，前缀部分的 KV Cache 只需计算一次，后续请求直接复用。

⁶视频画面时间区间：00:23:50-00:24:10。

其实现基于 Hash 的 Block 级别 KV Cache 复用：每个 Block 的 KV 张量计算一个 Hash 值，当新请求的某个 Block 的 Hash 与缓存中的 Block 匹配时，直接复用该 Block 而无需重新计算。

4.4 本章小结

KV Cache 是 Transformer 自回归生成中必不可少的优化，但显存管理一直是工程挑战。Page-dAttention 通过分页式内存管理实现了近乎零浪费的显存利用率，是 vLLM 框架的立身之本。前缀缓存在此基础上进一步通过跨请求共享减少重复计算。这两个技术共同构成了 vLLM 内存效率的核心竞争力。

5 分布式并行

5.1 集合通信基础

分布式计算中，多个 GPU 之间的数据交换依赖于集合通信（Collective Communication）。集合通信是机器学习集群中实现分布式训练和推理系统的基础，本质上是一个进程组中所有进程都参与的全局通信操作。

常见的集合通信算子：

- **Broadcast**（广播）：单点到多点，将一个 GPU 的数据发送给所有 GPU。不改变 Shape。
- **Scatter**（发散）：单点到多点，将数据分片发给不同 GPU。Shape 发生变化（变小的维度按 GPU 数量等分）。
- **Gather**（收集）：多点到单点，多个 GPU 的数据汇总到一个 GPU。Shape 会扩大。
- **Reduce**（规约）：多点到单点，多个 GPU 的数据经运算（如求和）后汇总到一个 GPU。Shape 不变（Reduced）。
- **All-Reduce**：所有 GPU 都参与规约，所有 GPU 都得到最终结果。最常用的分布式通信原语。
- **All-to-All**：每个 GPU 向其他 GPU 发送数据，同时从其他 GPU 接收数据。主要用于专家并行（EP）中的 token 路由。

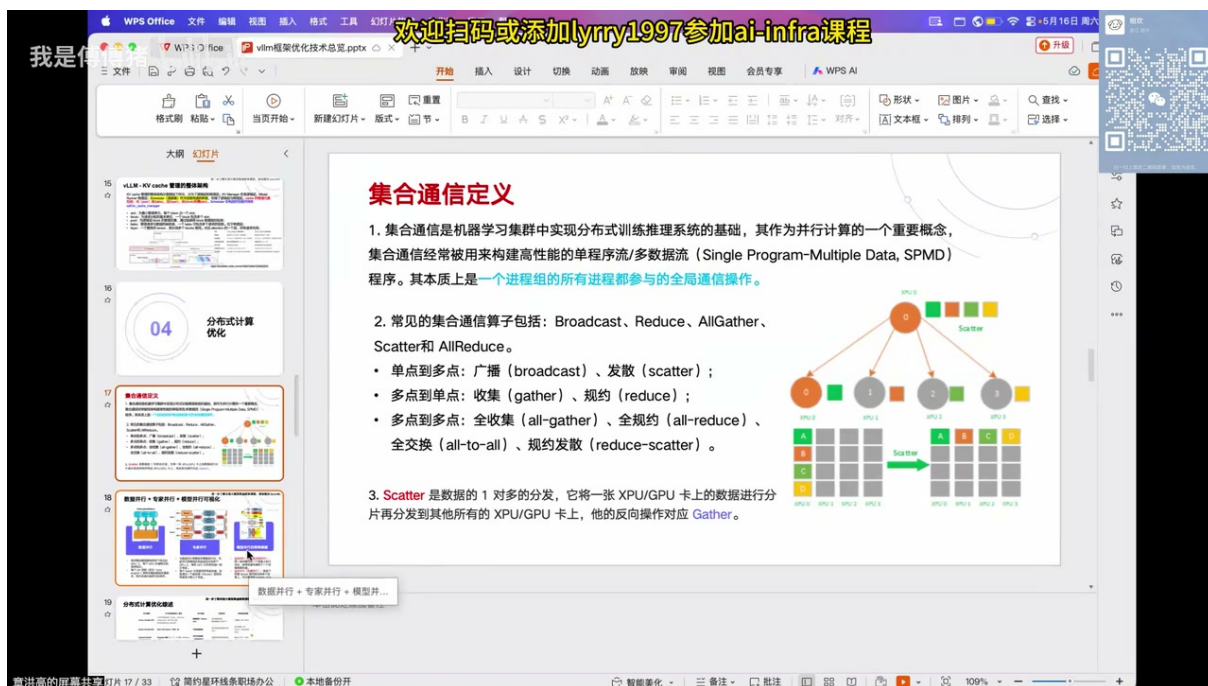


图 7: 常见集合通信算子示意⁷

5.2 数据并行（Data Parallelism, DP）

数据并行是最简单的分布式策略：将完整模型复制到每个 GPU，每个 GPU 独立处理不同的请求批次。

DP 的本质

数据并行不属于模型内部的切分，而是请求级别的并行。如果有 100 个请求和 DP=10，则每个 DP Rank 处理约 10 个请求。DP 不会引入额外的通信开销（因为没有模型切分），其核心作用是提升吞吐量，而非加速单个请求。

DP 与 TP 的配合：以 DP=2、TP=4 为例，总共 8 个 GPU。系统启动 2 个 Engine Core 进程（2 个 DP Rank），每个 Engine Core 内部通过 ZMQ 与前端通信获取不同请求，各自管理 4 个 GPU Worker（TP=4）。

三种负载均衡模式：

- **内部负载均衡：**由 vLLM 框架内部基于各 DP Rank 的负载状态做请求分发。
- **外部负载均衡：**由上层流量调度层（如网关、负载均衡器）基于各 DP Rank 暴露的 Metrics（如 Running/Waiting 请求数）做分发。
- **混合模式：**结合内部和外部负载均衡，vLLM 同时提供选择权。

⁷ 视频画面时间区间：00:27:50-00:28:10。

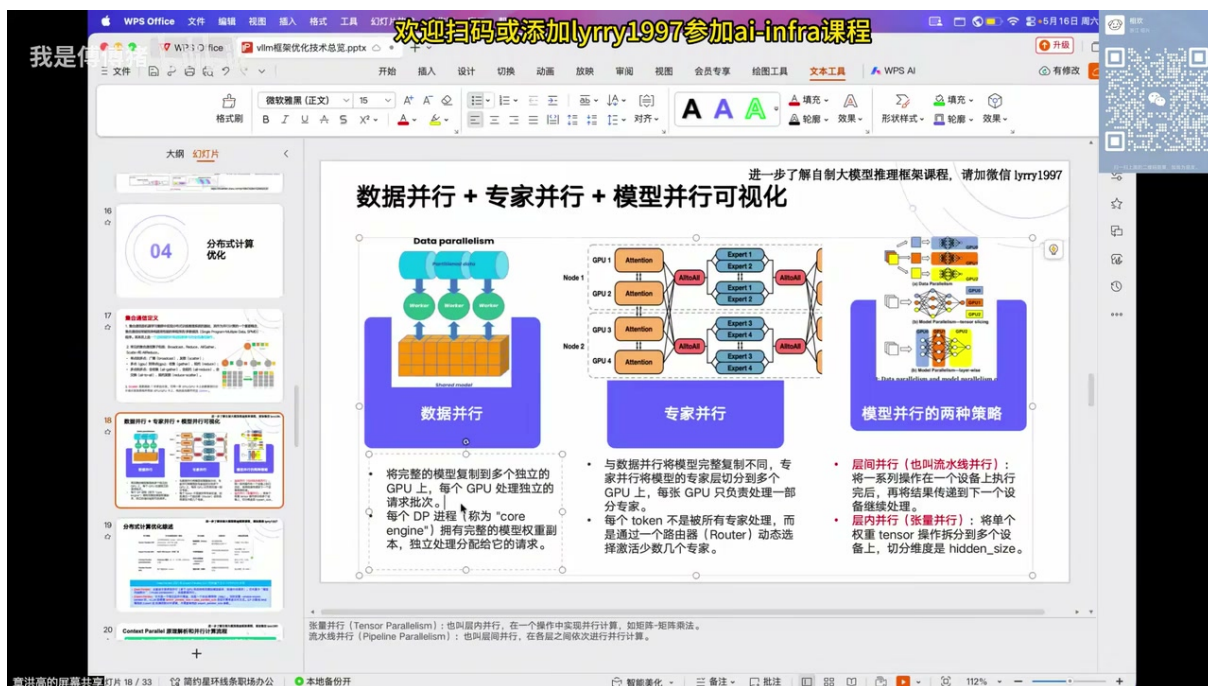


图 8: 数据并行的请求分发与多副本结构⁸

5.3 专家并行 (Expert Parallelism, EP)

专家并行是专为 MoE (Mixture of Experts) 模型设计的并行策略。与 DP 不同, EP 会切分模型: 每个 GPU 只负责一部分专家的计算。

EP 计算流程:

1. **输入准备:** 输入 `hidden_states`, 形状为 `[num_tokens, hidden_dim]`。
2. **Gate/Router:** Router 层决定每个 token 被分配到哪些专家。
3. **All-to-All Dispatch:** 根据路由结果, 将 token 分发到对应专家所在的 GPU。
4. **专家计算:** 各 GPU 对自己负责的专家做前向计算。
5. **All-to-All Combine:** 将计算结果回传到 token 原始所在的 GPU。

EP 会引发两次 All-to-All 通信: 一次 Dispatch (分发 token), 一次 Combine (回传结果)。

⁸视频画面时间区间: 00:29:50-00:30:10。

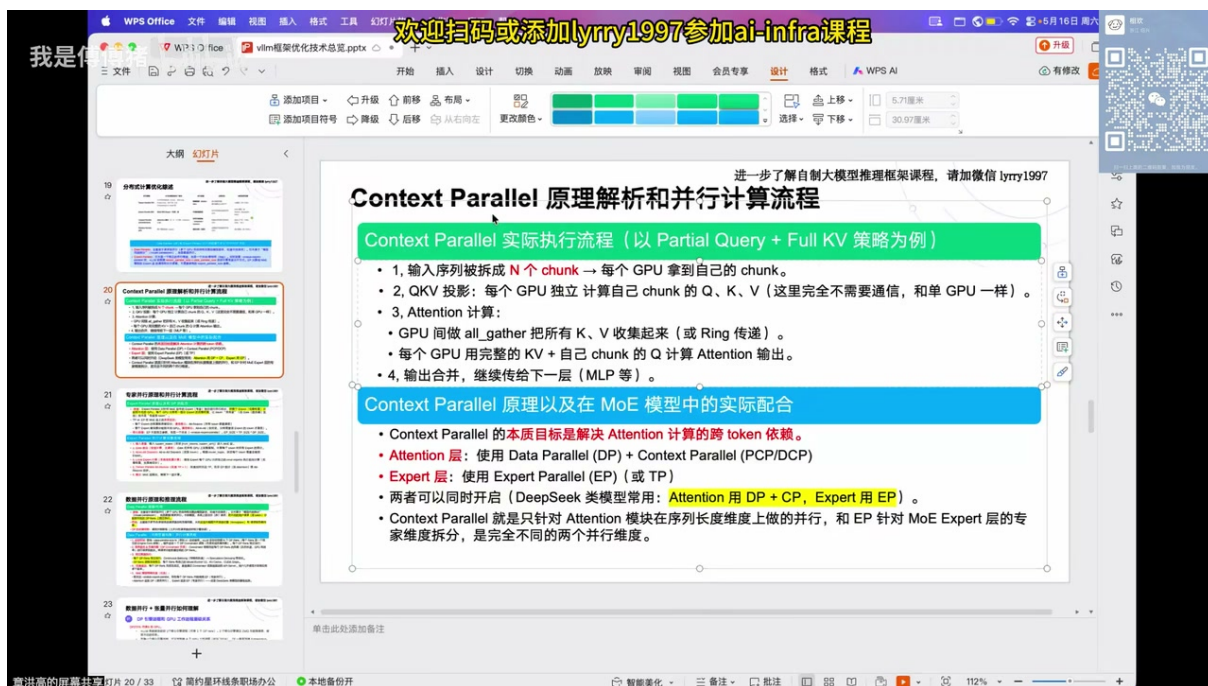


图 9: 专家并行的 Token 路由与 All-to-All 通信⁹

5.4 张量并行 (Tensor Parallelism, TP)

张量并行将单个权重矩阵按 `hidden_size` 维度拆分到多个 GPU 上并行计算，属于层内并行。

TP 的通信模式：TP 通常采用“先列后行”的策略——

- 对于 Attention 的 QKV 投影：按列拆分权重，各 GPU 独立计算后通过 All-Reduce 汇总。
- 对于输出投影 (O 矩阵)：按行拆分，各 GPU 部分结果直接拼接。

TP 的切分维度：一般沿 `hidden_size` 维度切分，与 DP (沿 `batch` 维度切分) 和 EP (沿 `num_experts` 维度切分) 形成互补。

⁹视频画面时间区间：00:33:50–00:34:10。

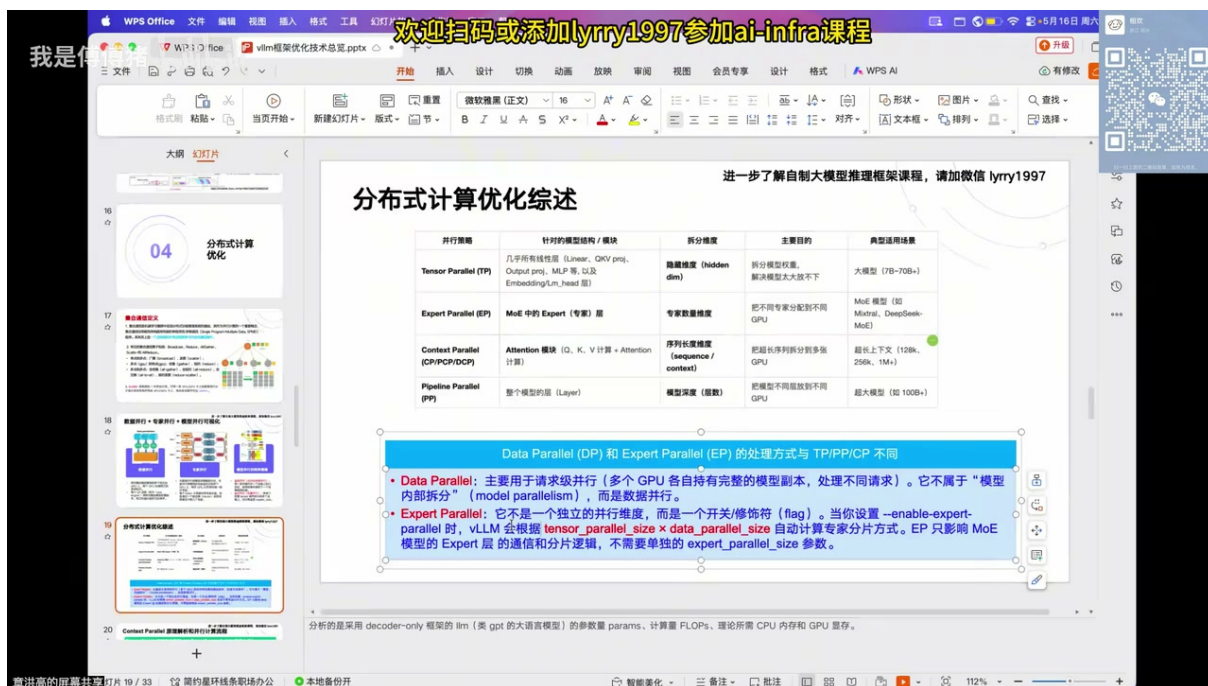


图 10: 张量并行的切分与通信模式¹⁰

TP vs EP 的通信差异

两者的通信模式有本质区别:

- TP 使用 All-Reduce 通信, 一般“先列后行”。
- EP 使用 All-to-All 通信, 涉及两次数据交换 (Dispatch + Combine)。
- 在 vLLM 中, EP 的 TP size 不是一个独立参数——它会根据 total batch size 和 DP size 自动推导。

5.5 上下文并行 (Context Parallelism, CP)

上下文并行是为了解决超长序列推理而设计的并行策略。其核心思想是将输入序列按 token 维度切分成多个 Chunk, 每个 GPU 独立计算自己 Chunk 的 QKV, 然后在 Attention 计算时通过 All-Gather 获取完整的 KV。

CP 执行流程:

1. 将输入序列拆成 N 个 Chunk, 每个 GPU 获得一部分 Token。
2. 每个 GPU 独立计算自己 Chunk 的 Q、K、V。
3. Attention 计算前执行 All-Gather, 每个 GPU 获得所有 GPU 的完整 K 和 V。
4. 每个 GPU 用完整 KV + 自己 Chunk 的 Q 做 Attention 输出。
5. 合并输出传递给下一层。

¹⁰ 视频画面时间区间: 00:31:50-00:32:10。

并行组合：CP 可以与其他并行策略组合。例如对于 MoE 模型，Attention 层使用 DP + CP，专家层（FFN/MoE）使用 EP。不同的 module 可以使用不同的并行策略，这是 vLLM 并行框架灵活性的体现。

5.6 本章小结

分布式并行是 vLLM 支持大规模模型和大批量推理的核心能力。DP 提升吞吐、TP 加速单层计算、EP 解决 MoE 模型的通信问题、CP 处理超长上下文。这四种并行策略可以灵活组合，根据模型结构和硬件拓扑选择最优配置。理解和区分它们的切分维度（DP 沿 batch、TP 沿 hidden_size、EP 沿 num_experts、CP 沿 sequence_length）是掌握 vLLM 分布式部署的关键。

6 量化压缩

量化压缩通过降低权重和激活的数值精度来减少显存占用和加速计算。在 vLLM 中，量化涉及权重、激活和 KV Cache 的全链路优化。

6.1 量化方法概览

- **GPTQ**：权重量化方法，使用基于 Hessian 矩阵的 Optimal Brain Quantization 思想，只量化权重，激活不做量化。
- **AWQ** (Activation-aware Weight Quantization)：权重量化方法，基于激活分布来确定每通道的重要性和量化参数。同样只量化权重。
- **SmoothQuant**：通过平滑因子将激活的量化难度转移到权重上，实现 W8A8（权重和激活都做 INT8 量化）。
- **FP8**：原生 FP8 精度，支持权重 + 激活 + KV Cache 的全链路量化。讲者认为 FP8/FP4 量化是未来的趋势——只要硬件支持，之前用 AWQ/GPTQ 这种 INT 量化是没办法中的办法。

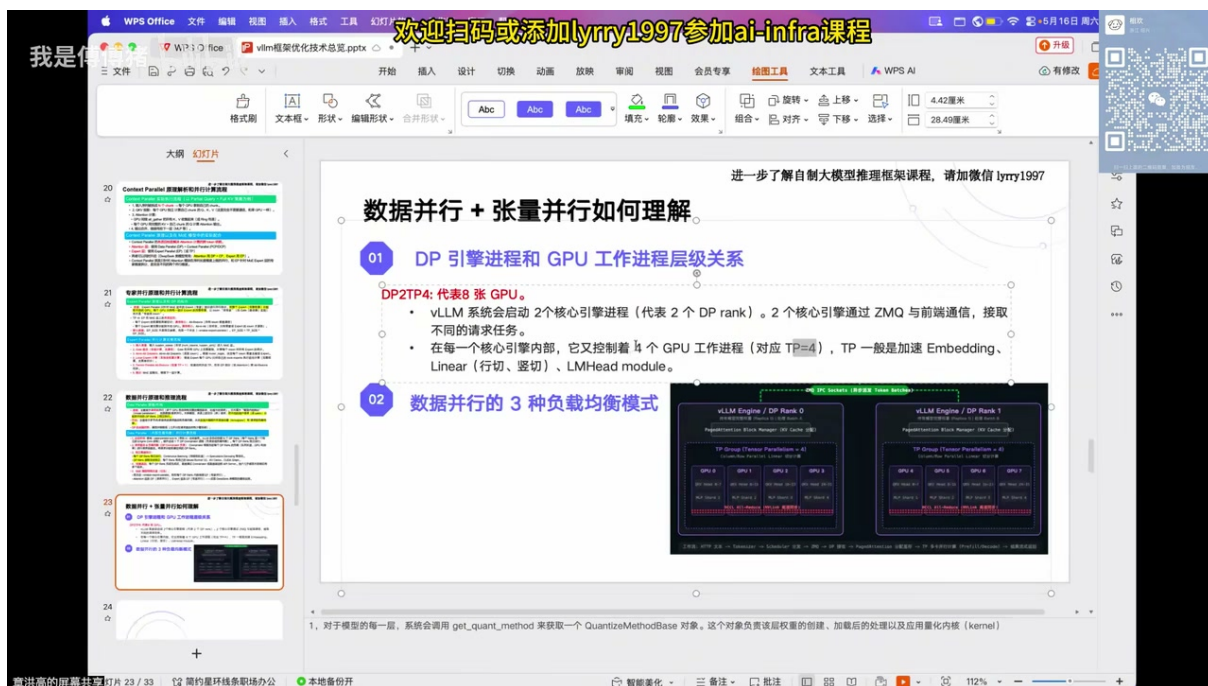


图 11: vLLM 量化方法汇总表¹¹

6.2 vLLM 量化设计原理

vLLM 的量化模块采用模块化设计，每种量化方法都是独立的实现（独立的类和独立的文件）。量化代码位于 `model_executor/layers/quantization/` 目录下：

- `fp8.py`: FP8 量化实现
- `gptq.py`: GPTQ 量化实现
- `awq.py`: AWQ 量化实现
- `compressed_tensors_quant.py`: 压缩张量量化

FP8 量化的两条主线：

1. **离线量化**：已知 Checkpoint 的 FP8 序列化 (`serialize=True`)，直接加载预量化的 FP8 权重。走 `FP8LinearMethod` 或 `FP8MoEMethod`。
2. **在线量化**：原始权重为 BF16/FP16，加载时动态量化为 FP8。走 `FP8OnlineLinearMethod`。

量化加载流程（以 FP8 为例）：

1. **创建量化配置**：模型加载时创建 `QuantizationConfig`（如 `FP8Config`）。
2. **获取量化方法**：通过 `get_quant_method()` 根据 Layer 类型分发到对应量化方法（Linear 层用 `FP8LinearMethod`，MoE 层用 `FP8MoEMethod`）。
3. **创建量化权重**：分配量化权重的内存空间，包括 `weight`、`weight_scale` 等参数。

¹¹ 视频画面时间区间：00:38:50–00:39:10。

4. **权重后处理**：通过 `process_weights_after_loading()` 完成权重的量化转换。
5. **Apply 推理**：在实际推理时通过 `apply()` 执行量化计算。如果输入不是 FP8，需要先对激活做动态量化（静态或动态），再进行 W8A8 矩阵乘法。

6.3 量化粒度

量化粒度从大到小排列：

- **Per-Tensor**：整个权重张量共用一个量化 Scale。粒度最大，精度最低。
- **Per-Token**：按 token 维度，每个 token 独立计算量化参数。
- **Per-Block**：按 Block Size 将权重分块，每个块独立计算量化参数。
- **Per-Group**：将权重的输出通道按 Group 分组（Group Size 一般为 64 或 128），每个组共用 Scale 和 Zero Point。AWQ 即为 Per-Group 量化。
- **Per-Channel**：每个输出通道独立一个量化参数，粒度最细。

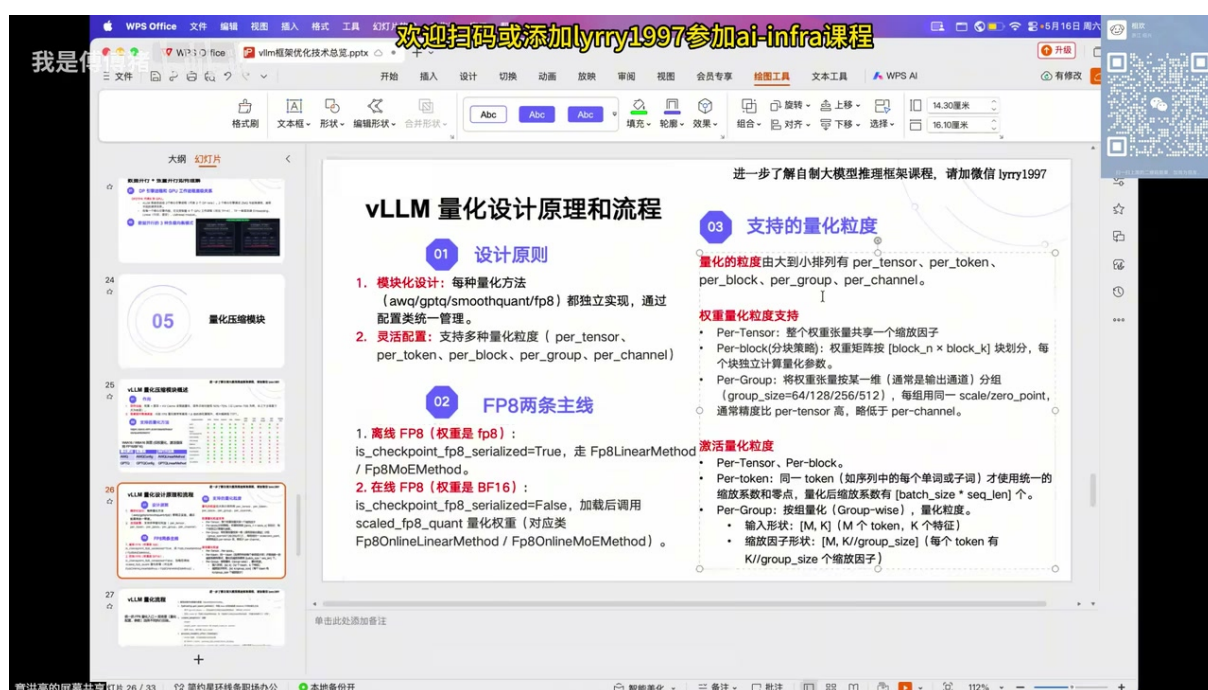


图 12: 量化粒度从大到小的对比¹²

量化粒度与精度的权衡

量化粒度越大（如 Per-Tensor），所用 Scale 参数越少，压缩率越高，但精度损失越大。量化粒度越小（如 Per-Channel），精度越高但额外存储的 Scale 参数也越多。实际工程中需要在压缩率和精度之间做权衡。

¹² 视频画面时间区间：00:43:50–00:44:10。

6.4 本章小结

vLLM 的量化模块通过模块化设计支持多种量化方法，FP8 是当前主流趋势（DeepSeek 最早采用 FP8 训练，后续模型多原生支持）。量化加载流程遵循“创建配置 → 获取方法 → 创建权重 → 后处理 → Apply”的标准路径。量化粒度的选择直接影响精度和压缩率的平衡。

7 投机解码

7.1 投机解码的动机

大语言模型的自回归解码分为两个阶段：Prefill（预填充）和 Decode（解码）。Decode 阶段一次只能生成一个 token，这种逐 token 的串行解码导致：

- 内存带宽瓶颈（Memory-bound）：每次 Decode 步的计算量不大，但需要加载整个模型的权重，内存带宽成为瓶颈。
- 串行效率低：生成 K 个 Token 需要 K 次串行模型调用。

投机解码的核心思想：用一个更轻量的 Draft Model（草稿模型）一次性生成多个候选 Token，然后用大模型一次并行验证这些候选 Token 的正确性。这等价于将串行解码转变为并行解码——一次推理生成的 Token 越多，模型总 Forward 调用次数越少。

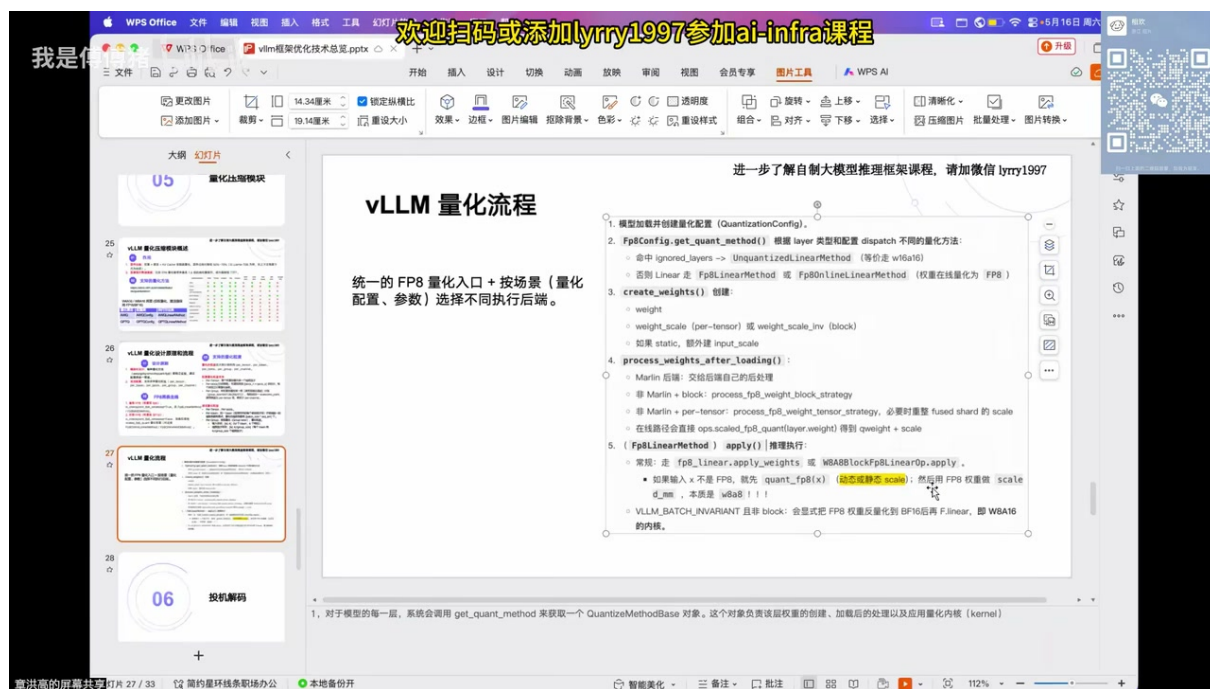


图 13: 自回归解码 vs 投机解码的对比¹³

7.2 Medusa 方案

Medusa 是一种单模型的投机解码方案：在主 LLM 顶部增加多个解码头（Medusa Head），直接预测 Next Token。不需要额外的小模型。

¹³ 视频画面时间区间：00:47:50–00:48:10。

WPS Office

文件 编辑 视图 插入 工具 幻灯片放映

我是佛系程序员

WPS AI

开始 插入 设计 切换 动画 放映 审阅 视图 会员专享 有修改

格式刷 粘贴 当页预览 新建幻灯片 版式 目录

B I U A S X P 对开

形状 图片 查找 文本框 排列 选择

大纲 幻灯片

06 投机解码

07 投机解码的模型结构图

08 投机解码的并行推理的基本原理

09 vLLM 支持的投机解码方案

vLLM 支持的投机解码方案

无 Draft 小模型的多头并行解码

Medusa 提出了一种 单模型 (One-Model) 并行解码方案：在主干 LLM 顶部增加多个解码头 (Medusa Heads)，直接做 Next-Next-Token 预测，代替额外 Draft 小模型。

Medusa 模型结构代码 (简略版本)，
• 权重初始化为 0:

- 刚开始 linear(x) 输出 ≈ 0
- $\text{act}(0) \approx 0 \rightarrow \text{forward}(x) \approx x$
- 即整个 ResBlock 初始为 近似恒等映射
- 训练过程中，linear.weight 从 0 逐步学到有意义的变换，形成轻量 decoder layer。

```
<pre><code># Python
from typing import List, Tuple
import torch
import torch.nn as nn
import torch.nn.functional as F
from transformers.models.gpt2.modeling_gpt2 import GPT2Block, GPT2LMHeadModel

class MedusaLanguageModel(nn.Module):
    """The Medusa Language Parallel Model"""
    def __init__(self, num_heads: int, lm_head_model: GPT2LMHeadModel):
        super().__init__()
        self.lm_head_model = lm_head_model
        self.num_heads = num_heads
        # Create parallel heads
        self.parallel_heads = nn.ModuleList()
        for i in range(num_heads):
            head = GPT2LMHeadModel(lm_head_model.config.vocab_size, lm_head_model.config.hidden_size)
            self.parallel_heads.append(head)

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, List[torch.Tensor]]:
        """Forward pass of the Medusa model. Returns the main output and a list of speculative outputs from the heads."""
        main_output, _ = self.lm_head_model.forward(x)
        speculative_outputs = []
        for head in self.parallel_heads:
            speculative_outputs.append(head.forward(x))
        return main_output, speculative_outputs</code></pre>
```

单点此添加备注

本地备份开

智能美化 备注 批注 112%

7.3 Eagle 方案

Eagle 的模型结构:

- ¹⁴视频画面时间区间：00:49:50-00:50:10。

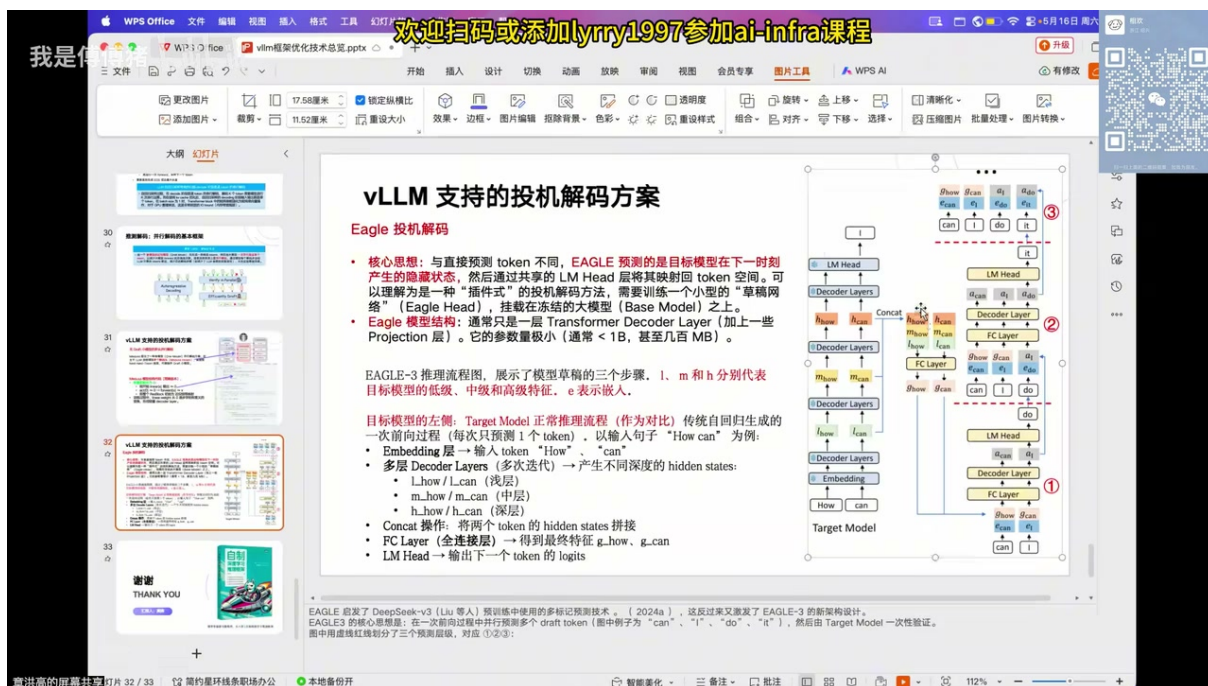


图 15: Eagle 草稿模型的结构与特征提取¹⁵

7.4 MTP 方案

Multi-Token Prediction(MTP)是 DeepSeek V3 和千问 3.5/3.6 等最新模型原生支持的投机解码方式。与 Medusa 和 Eagle 需要额外模块不同，MTP 是模型训练时就内建的 Next Token Prediction 能力。

讲者指出：DeepSeek 和千问代表了趋势——未来更多模型会原生支持 MTP，独立的 Draft Model 方案（如 Eagle）可能不再是主流方向。在生产环境中，MTP 在多卡部署时与主模型一起做并行，无需额外的部署考量。

投机解码的工程现实

- 加速比取决于接受率：接受率越高，加速比越大。工业界平均接受率在 2.x 左右，加速比约 2-3 倍。
- 简单任务（如生成 Python 代码）接受率可达 6+，复杂任务较低。
- 端侧场景对投机解码的需求更强（功耗和延迟敏感）。
- 高并发场景下，Batch Size 已经很大时，投机解码未必能带来额外收益（因为 GPU 计算已经饱和）。

7.5 本章小结

投机解码本质上是将串行解码变为并行解码，通过增加每次 Forward 的 Token 产出量来减少总调用次数。Medusa 和 Eagle 是两种插件式方案，而 MTP 是原生支持的未来方向。接受率是衡量投机解码效果的核心指标，与加速比直接相关。

¹⁵ 视频画面时间区间：00:51:50-00:52:10。

8 AI-Infra 学习路径与职业发展（答疑精选）

本章内容来自视频后半程（第 54 分钟至结尾）的答疑与交流环节。讲者结合自身经验和行业观察，回答了学员关于学习方法、转行路径、面试准备和职业规划的问题。以下为精选摘要。

8.1 从零开始学 AI-Infra

Q：AI-Infra 怎么从零开始学？

- **优先了解最新模型**：通过 DeepSeek 和千问这两个代表性开源模型了解 AI-Infra 的整体图景。作为 Infra 工程师，要跟着模型走才不会掉队。
- **选择技术方向**：推理 Infra 的技术栈整体排序（从底层到上层）：驱动/运行时 → 编译器 → 算子层 → 框架层 → 平台调度层。讲者个人推荐做推理框架。
- **人行成本最低的路径**：DP 相关的负载均衡和服务调度——对于有后端开发经验的工程师，迁移成本低，天然契合。
- **关注门槛高的方向**：分布式并行（大规模 EP/TP/DP 部署，解决精度和性能问题），经验越稀缺越值钱。

讲者推荐的技术学习路线

从门槛最低的后端服务调度切入，逐步深入框架内核（数据并行、调度器、量化模块），最终掌握分布式并行等核心技术。同时始终跟踪 DeepSeek 和千问的最新模型变化，这是 Infra 工程师的“行业脉搏”。

8.2 转行与面试准备

Q：社招转推理框架岗，需要什么程度的准备？

- **简历写法**：写清楚自己负责的具体模块，不要写得太大。如果你写的比较特定，面试官不会多问你没写过的东西。
- **开源贡献（PR）**：社招转岗最重要的一条路就是给开源框架提 PR。从小 Feature 或 Bug Fix 做起，不需要一上来就做大 Feature。只要有能力提 PR，一般面试不会有问题。
- **面试深度**：面试根据简历来——你写了了解 PagedAttention，就会问 PagedAttention 的原理。写了分布式并行，就会问张量并行为什么“先列后行”，通信量是多少。**没写的不问，写了就要准备好被深问。**
- **手写代码**：框架岗的代码考察偏向实际工作场景：手写 Attention Module (Python)、推导 Online Softmax 公式、手写 All-Reduce 的通信逻辑。不考 LeetCode 算法题，考的是和岗位相关的代码。

8.3 行业现状与趋势

- **岗位分层**：推理框架岗是一个“补位”逻辑——团队缺哪块的人就招哪块的。不需要你懂所有模块，但你需要对你负责的模块非常精通。

- **AI Coding 的普及**：讲者提到自己今年已经很少手写代码，基本都用 Codex 等 AI 编程工具。但对于框架开发，需要把设计目标和模块边界说清楚效果才好。建议学会写 Skill/Prompt。
- **语言选择**：推理框架仍以 Python 为主，但需要能看懂和写一些 C++（尤其是算子开发）。语言不是关键，会用中文写 Prompt 最重要。
- **工作节奏**：如果在芯片公司，业务以模型支持为主——跟着 DeepSeek 和千问等新模型走，支持新 Feature（量化、算子、模型结构搭建）。如果在互联网公司，更偏向业务驱动的快速迭代。
- **SGLang vs vLLM**：两者已有重叠，融资后差异性可能更明显。芯片公司关注 vLLM 更多（SGLang 对国产芯片支持较弱）。

8.4 本章小结

AI-Infra 的学习路径应紧跟模型趋势（DeepSeek、千问），从自身背景最契合的方向切入（如后端转服务调度、算子转量化/编译）。面试的核心原则是“简历写什么就问什么”——把有限的时间花在让你简历上最亮眼的那个模块上，把它搞透。

9 总结与延伸

9.1 讲者结尾讨论

视频结尾，讲者与主持人互相道别，约定下一次分享将交换角色——由主持人主讲 vLLM 的另一个主题，讲者以“学生”身份参与。这反映出 AI-Infra 社区的务实文化：分享与学习交替进行，每个人既当老师也当学生。讲者也坦言近期状态不佳，但强调“也不能怪公司”——这种坦诚让人感受到一线工程师的真实节奏。

9.2 核心知识综合

本视频的核心教学内容可以浓缩为以下几条主线：

1. vLLM 的架构灵魂：分层抽象 vLLM 的六层架构（LLM → Engine Core → Scheduler → Executor → Worker → Model）是为了同时支撑 DP、TP、EP、调度优化等多种特性而设计的。每一层都有明确的职责边界，但也带来了系统复杂度。未来这些抽象能否精简，仍是一个开放问题。

2. 显存效率的核心：PagedAttention + 前缀缓存 PagedAttention 是 vLLM 的立身之本——它将操作系统的分页思想引入 KV Cache 管理，实现了近乎零浪费的显存利用率。前缀缓存在此基础上通过 Hash 的 Block 级复用进一步减少跨请求的重复计算。

3. 显存与计算的量化压缩 量化从权重扩展到激活、再到 KV Cache 的全链路优化。FP8 已经成为主流趋势，AWQ/GPTQ 等 INT 量化方法将逐渐退居补充角色。量化粒度（Per-Tensor → Per-Group → Per-Channel）的选择直接影响精度和压缩率的平衡。

4. 自回归解码的效率革命：投机解码 从 Medusa 的多个预测头，到 Eagle 的 Hidden State 预测，再到 MTP 的原生多 Token 预测——投机解码正从“插件式”走向“原生”。DeepSeek 和千问的方向代表了未来：MTP 会成为标配，独立的 Draft Model 方案可能不再是主流。

5. 分布式并行的灵活组合 DP、TP、EP、CP 四种并行策略可以在不同 Module 上灵活组合。核心在于理解各自的切分维度（DP 沿 batch、TP 沿 hidden_size、EP 沿 num_experts、CP 沿 sequence_length）和通信模式（All-Reduce vs All-to-All）。

9.3 实践要点

- **学习 vLLM 源码的起点**: 从 `examples/` 目录运行最简单的离线推理开始, 然后自上而下追踪代码流: `LLM.generate()` → `Engine` → `Scheduler` → `Executor` → `Worker` → `Model`。不要一上来就钻进 `model_executor/layers/` 目录。
- **跟踪最新动态**: 每隔一段时间浏览 Release Notes 和 Blog。vLLM 更新极快, 旧文章和旧代码可能已经过时。
- **理解通信开销**: 分布式部署时, 优先考虑 DP + EP 模式 (通信开销低), TP 适合 Prefill 阶段 (计算密集), CP 按需开启。
- **PD 分离的价值**: 并非所有模型都需要 PD 分离——模型规模不够大或单次推理时没必要。但对于 671B 级别的 DeepSeek V3/V4 等大模型, PD 分离是解决 OOM 和提升吞吐的有效手段。
- **转行入局的策略**: 从小 PR 开始积累开源贡献, 简历上聚焦于一个你最擅长的模块, 面试时把它讲透。面试官是按“补位”逻辑招人的, 不需要你全栈精通。

9.4 拓展阅读

- vLLM 官方文档: <https://docs.vllm.ai/>
- vLLM Blog (PagedAttention 深度解读): <https://blog.vllm.ai/>
- vLLM GitHub 仓库: <https://github.com/vllm-project/vllm>
- PagedAttention 论文: Efficient Memory Management for Large Language Model Serving with PagedAttention (SOSP 2023)
- DeepSeek V3 技术报告: [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- 千问 3 官方博客: <https://qwenlm.github.io/blog/qwen3/>